

FRONTEND DEVELOPER

SOHYEON JEON

이름 전소현 이메일 tamizy@naver.com 깃허브 @laven-ia

시티아이랩(주) 기업부설연구소

Frontend Developer | 2025.05 - 현재

UI/UX 설계 및 구현부터 영상 스트리밍, 실시간 데이터 처리 및 Canvas·Chart·Three.js 기반 시각화, Electron 현장 배포까지 프론트엔드 전반을 담당했습니다.

01 전기차 화재 감지 시스템

Frontend 1명, Backend 3명 | 2025.05 – 현재

| 프로젝트 개요

- 가시광·열화상 카메라와 이기종 센서를 통합한 실시간 화재 감지·관제 시스템
- 영상·온도 데이터를 기반으로 ROI 관리, 이상 온도 알람 제공
- AI 영상 분석 결과를 통해 화재·연기·사람 탐지 결과 시각화

| 담당 역할 : Frontend Developer

- UI/UX 설계 및 개발
- 실시간 영상·센서 데이터 통합 관제, ROI 설정, 이상 온도 알람 및 탐지 이력 기능 구현
- Canvas 기반 ROI 편집과 AI 탐지 객체 BBox 실시간 시각화 구현
- RTSP → HLS 영상 스트리밍 및 Electron 기반 현장 통합 배포 구조 구축
- JavaScript → TypeScript 마이그레이션 및 프론트엔드 구조 개선
- 일부 API 유지보수 (FastAPI, Python)

| 24시간 관제 환경의 화이트 스크린 및 메모리 증가 원인 추적

상시 가동되는 Electron 기반 관제 애플리케이션에서 메모리가 지속적으로 증가하고, 24시간 이내 주기로 화면이 하얗게 멈추는 장애가 발생했다.

01. 원인 분석 및 검증

Chrome DevTools의 Heap Snapshot·Allocation Timeline으로 JavaScript 객체의 생성·해제 추이와 Retainer 경로를 확인하고, 비동기 작업과 외부 리소스의 생명주기를 점검했다. JavaScript Heap의 누수 요소를 정리한 이후에도 프로세스 메모리가 계속 증가하여, Windows 작업 관리자의 PID별 활성 개인 메모리와 커밋 크기를 추적하며, **Electron renderer 프로세스로 분석 범위를 확장**했다.

JS Heap 분석만으로 원인을 찾기 어려워지자, 문제가 발생할 가능성이 있는 렌더링 컴포넌트를 하나씩 비활성화하며 기능별 분리 실험을 진행했다.

- 모든 렌더링 컴포넌트를 비활성화하고, WebSocket 데이터 수신·저장만 유지했을 때는 메모리 증가하지 않아 렌더링 경로를 원인 영역으로 특정
- 컴포넌트를 단계적으로 활성화한 결과, ECharts의 반복적인 setOption() 실행 경로까지 범위를 축소
- axis·series·animation을 분리해 반복 검증한 결과, **실제 시각을 동적으로 계산하는 X축 formatter가 반복 실행되는 조건에서 renderer memory가 지속적으로 증가하는 것을 확인**

렌더링 빈도를 낮추는 방법은 메모리 증가 속도만 늦출 뿐이라고 판단했다. 24시간 상시 운영 환경에서는 장애 시점을 늦추는 것보다 누적 자체를 제거하는 것이 필요했기 때문에 **동적 시간축 렌더링 구조 자체를 변경**했다.

02. 구조 개선

AI를 활용해 원인에 대한 기술적 가설과 개선 대안을 확장하고, 실제 재현 조건과 renderer memory 변화를 기준으로 적용 여부를 판단했다.

- 실제 시각에 따라 변경되던 X축을 **고정 상대 좌표 범위로 전환**하고 데이터 좌표만 최신 시각 기준으로 변환
- 실제 시간 라벨을 ECharts Canvas에서 **React DOM으로 분리**해 고정 노드를 재사용하고 갱신 주기를 초 단위로 제한
- 비동기 요청 및 WebSocket · Worker · Timer · RAF · Observer cleanup 정비, ImageBitmap.close()를 통한 네이티브 리소스 해제
- 실시간 데이터에 **고정 크기 rolling window**, 열화상 프레임 처리에는 **최신 프레임 우선 backpressure를 적용**해 데이터와 비동기 작업의 무제한 누적 방지

03. 장애 복구 구조

메모리 문제 해결과 별개로 장시간 무인 운영 중 발생할 수 있는 장애에 대비해 복구 구조를 구축했다.

- WebSocket 재연결을 최대 대기 시간을 둔 지수 백오프 방식으로 변경
- React · Electron 오류를 통합 수집해 IPC를 통해 백엔드로 전달
- 렌더링 오류 시 fallback UI와 지연 새로고침 적용
- renderer 종료·응답 없음 상태를 main process에서 감지해 단계적 reload 및 BrowserWindow 재생성
- localStorage 기반 상태 영속화로 renderer 복구 후 기존 관제 상태 복원

04. 결과

- **24시간 미만 주기로 발생하던 화이트 스크린 현상 해소**
- **JavaScript Heap 및 Electron renderer memory의 지속적인 우상향 안정화**
- renderer · WebSocket · 백엔드 장애 발생 시 자동 복구 및 관제 상태 복원 구조 확보
- 프론트엔드 오류 수집 체계를 구축해 장시간 운영 환경의 장애 진단 범위 확대

프론트엔드 API 계층 및 데이터 관리 구조 재설계

메인 페이지에 **여러 기능 영역의 API 호출과 서버 상태 관리 로직 대부분이 집중**되어 있었다. 각 API의 fetch와 상태를 주로 사용하는 컴포넌트를 기준으로 정의하고 props로 전달하는 구조였기 때문에, 기존 데이터를 다른 컴포넌트에서도 사용해야 할 경우, **상태와 fetch 로직을 상위 컴포넌트로 이동하고 다시 props를 하위 계층까지 전달**해야 했다.

기능과 API가 추가될수록 데이터 사용 위치에 따라 기존 컴포넌트 구조까지 변경해야 했고, AbortController·timeout·예외 처리·snake_case → camelCase 변환 등의 코드도 API마다 반복되고 있었다.

01. 구조 분석 및 설계

개별 fetch 코드를 정리하는 것만으로는 이러한 문제를 해결하기 어렵다고 판단했다. 핵심 문제를 **서버 상태의 소유 위치가 컴포넌트 트리에 종속되어 있다는 점**으로 보고, API 호출과 서버 상태를 UI 컴포넌트에서 분리하는 방향으로 구조를 재설계했다.

책임을 **HTTP 통신 → API 응답 변환 → 서버 상태 관리 → UI**로 분리하고, polling · 캐싱 · mutation 이후 재조회가 반복되는 특성을 고려해 직접 상태를 비교·전달하는 대신 **TanStack Query의 Query Cache를 중심으로 서버 상태를 공유하는 구조를 적용**했다.

02. API 및 서버 상태 관리 구조 개선

- httpClient에 timeout · 응답 parsing · 공통 에러 처리를 통합하고 HttpError를 통해 예외 처리 방식 표준화
- types/{domain}.api.ts → services → hooks/queries로 계층을 분리하고 서버 raw response와 프론트엔드 domain model 분리
- HTTP 호출과 snake_case → camelCase 변환을 service 계층에 캡슐화
- queryKey 기반 캐시를 통해 **컴포넌트 위치와 관계없이 동일 서버 상태를 공유하도록 변경**
- polling과 request deduplication을 Query 계층으로 이동하고, mutation 이후 invalidateQueries로 관련 서버 상태 재동기화
- structural sharing을 활용해 기존 JSON.stringify(prev) === JSON.stringify(next) 기반 직접 깊은 비교 및 상태 동기화 로직 제거

03. 결과

- **메인 페이지 기준 API 관련 코드 약 500줄 감축**
- 새로운 컴포넌트에서 기존 서버 데이터가 필요할 때, **상위 컴포넌트로 상태를 이동하고 props 전달 구조를 변경하던 작업 제거**
- fetch 함수·상태 setter의 prop drilling 제거
- 동일 서버 데이터를 컴포넌트별로 개별 요청·관리하던 구조를 Query Cache 기반으로 통합
- API 응답 타입·데이터 변환·에러 처리를 기능별 동일한 구조로 표준화

실시간 스트리밍 생명주기 및 송출 안정성 개선

RTSP 영상을 FFmpeg · MediaMTX를 통해 HLS/WebRTC로 제공하는 과정에서 **실제 시청자가 있는데도 스트림이 종료되어 영상이 재연결되는 문제가 발생**했다. 동시에 MediaMTX 로그에서 RTP packet loss와 processing error가 반복되어 장시간 시청 중 화면이 멈추는 문제도 발생했다.

01. 스트림 종료 원인 분석

기존에는 프론트엔드가 주기적으로 보내는 heartbeat를 기준으로 스트림 사용 여부를 판단하고, 일정 시간 ping이 없으면 FFmpeg를 종료했다.

그러나 브라우저 타이머 지연, 컴포넌트 생명주기, 요청 취소 및 서버 처리 지연 등으로 heartbeat가 정해진 시간 안에 도달하지 않을 수 있었다. 실제 MediaMTX 로그를 확인한 결과 **WebRTC reader가 is reading 상태임에도 ping timeout으로 스트림이 종료되는 패턴을 확인**했다.

이에 프론트 heartbeat를 실제 시청 여부의 기준으로 사용하는 대신, **MediaMTX가 직접 감지하는 reader 연결 상태를 스트림 생명주기의 기준으로 변경**했다.

02. 스트림 생명주기 재설계

- /register-stream에서 RTSP 정보를 사전 등록하고, 실제 시청 또는 해상도 조회 시에만 FFmpeg를 기동
- MediaMTX의 runOnDemand·runOnRead·runOnUnread 이벤트를 서버 상태와 연동해 reader 접속·해제를 추적
- 프론트 heartbeat 기반 활성 상태 판단을 제거하고 **실제 reader 존재 여부를 기준으로 스트림 유지·종료**
- 해상도 조회 시 스트림이 비활성 상태라면 등록된 RTSP 정보로 자동 기동하는 복구 경로 구성

이를 통해 스트림 등록 → 기동 → reader 감지 → 종료로 이어지는 생명주기를 하나의 흐름으로 정리하고, 프론트 상태와 실제 미디어 서버 상태가 불일치하면서 정상 스트림을 종료하던 문제를 제거했다.

03. 송출 안정성 개선

MediaMTX 로그에서 반복되는 RTP packet loss와 processing error를 분석하고, FFmpeg → MediaMTX 구간의 RTSP/RTP transport를 조정했다.

패킷 손실을 줄이기 위해 처음에는 **TCP transport를 우선 적용**했지만, 현장 RTSP 소스에 따라 TCP 연결 자체가 성립하지 않는 경우가 추가로 확인됐다. 이에 특정 transport만 강제하는 대신 **TCP 연결을 우선 시도하고, 실패하면 UDP로 fallback하는 방식으로 변경**해 패킷 손실 완화와 소스 호환성을 함께 고려했다.

또한 기존 Low-Latency HLS는 짧은 segment/part 단위로 동작하는 만큼 운영 환경의 네트워크 지터와 패킷 손실에 민감하다고 판단했다. 관제 시스템에서는 수 초 수준의 지연 감소보다 **영상이 장시간 안정적으로 유지되는 것을 우선**해 fMP4 기반 일반 HLS로 전환하고 segment duration을 조정했다.

04. 결과

- **실제 시청 중인 스트림이 heartbeat 누락으로 종료되던 문제 해소**
- 실제 reader 접속·이탈을 기준으로 스트림을 관리해 미시청 스트림 종료 판단 개선
- 스트림 등록 → 영상 시청 시 기동 → reader 감지 → 종료 흐름 일원화
- RTP packet loss 및 processing error 발생 빈도를 낮춰 HLS/WebRTC 시청 중 화면 멈춤과 재연결 현상 완화
- 저지연보다 장시간 **송출 안정성을 우선하는 관제 환경에 맞게 스트리밍 설정 재구성**

기술 스택

React · TypeScript · JavaScript · Tanstack Query · Canvas API · Echarts · Rest API · Websocket · MediaMTX · FFmpeg · Electron · Git · Jira · Figma

02 교통 신호 운영 분석 소프트웨어

Frontend 1명, Backend 2명 | 2025.07 – 2026.01

프로젝트 개요

- SUMO·Unity·React를 연동한 디지털 트윈 기반 실시간 교통 시뮬레이션 시스템
- 교차로 신호 최적화 전·후의 교통 흐름을 3D 시뮬레이션하고 성능 비교·통계 시각화 제공

담당 역할 : Frontend Developer

- UI/UX 설계 및 개발
- 교차로 선택, 모델 생성, 최적화 요청, 이력 조회 및 전후 성능 비교 화면 구현
- React에 Unity WebGL을 통합해 실시간 3D 교통 시뮬레이션 제공
- WebSocket 기반 FCD 데이터 수신 및 차량 위치 실시간 시각화 구현
- QGIS 기반 지형·건물 데이터 가공 및 Unity 시뮬레이션 환경 구축
- 수 GB 규모 FCD 데이터를 처리하는 Python 데이터 파이프라인 구현

고해상도 지도 Canvas 렌더링 병목 개선

고해상도 지도 위에 다수의 노드를 표시하는 관제 화면에서 드래그·줌 시 심각한 프레임 저하가 발생했다. 특히 줌 아웃 상태에서도 고해상도 원본 이미지를 매 프레임 축소하고, 지도와 인터랙션 요소를 동일한 Canvas에 다시 그리는 구조로 인해 불필요한 렌더링 비용이 반복되고 있었다.

01. 병목 분석 및 개선 방향

Chrome DevTools Performance로 렌더링 구간을 분석한 결과, JavaScript 실행보다 **고해상도 이미지의 반복적인 Painting이 주요 병목임을 확인**했다. 지도는 변경 빈도가 낮은 정적 리소스인 반면 드래그·줌 과정에서는 동일한 축소 연산이 반복되고 있었기 때문에, 매 프레임 원본을 리샘플링하는 대신 **자주 사용하는 해상도의 이미지를 사전 생성·캐싱해 재사용하는 방향으로 개선**했다.

02. 렌더링 구조 개선

- **정적 지도와 동적 인터랙션 요소를 Multi-Canvas 구조로 분리**해 변경이 필요한 레이어만 다시 렌더링
- 원본의 50%·25% 축소 이미지를 Offscreen Canvas에 사전 생성하고, 줌 배율에 따라 적절한 이미지를 선택하는 **다중 해상도 이미지 캐시** 적용
- PNG를 WebP(quality 75)로 변환해 초기 이미지 로딩 비용 축소

추가로 WebP 변환 과정에서는 이미지 해상도가 변경되면서 기존 노드 좌표와 지도 위치가 어긋나는 문제가 발생했다. 이를 원본 대비 해상도 비율을 기준으로 좌표와 offset을 동적으로 보정해 이미지 해상도가 변경되어도 기존 좌표 체계를 유지하도록 했다.

03. 결과 (Chrome DevTools Performance 기준)

- 지도 로딩 시간 약 10초 → 3초
- Painting Time 약 4,300ms → 170ms (약 96% 감소)
- Main Thread Long Task 51회 → 7회 (약 86% 감소)

대용량 FCD 처리 구조 개선 및 Unity WebGL OOM 해결

SUMO에서 생성되는 FCD(Floating Car Data) XML이 수 GB 규모로 증가하면서 전체 변환에 약 400초가 소요됐고, 변환된 데이터를 Unity WebGL에 모두 적재할 경우 **브라우저 메모리 한계로 OOM이 발생**했다.

01. 문제 분석 및 구조 변경

먼저 XML을 JSON으로 변환하고 시뮬레이션에 필요한 필드만 선별해 데이터 크기를 줄였다. 이를 통해 처리 시간은 약 400초에서 115초까지 단축됐지만 Unity WebGL의 OOM은 해결되지 않았다.

이를 통해 문제를 단순한 데이터 크기나 변환 성능이 아니라 **전체 데이터를 사전에 변환하고 클라이언트 메모리에 보관하는 구조 자체의 한계로 판단**했다. 시뮬레이션에서는 전체 FCD를 동시에 사용할 필요 없이 **현재 시점에 해당하는 데이터만 필요**하다는 특성에 주목해, **전체 적재 방식에서 필요한 시점의 데이터만 조회·전송하는 구조로 변경**했다.

02. 데이터 처리 구조 개선

- XML 전체 변환·적재 방식에서 **NDJSON 기반 스트리밍 처리 구조**로 전환
- 시뮬레이션 진행 시, **필요한 시점의 데이터만 WebSocket을 통해 Unity WebGL로 전달**
- 시뮬레이션 시간과 파일의 **byte offset을 매핑한 인덱스를 사전에 생성**
- 요청마다 대용량 파일을 처음부터 탐색하지 않고, byte offset으로 직접 이동해 **필요한 구간만 조회**
- Python으로 데이터 처리 파이프라인을 구현하고, Java 백엔드에서 이어서 활용할 수 있도록 처리 구조와 방식을 인계

03. 결과

- **FCD 처리 시간 약 400초 → 85초, 약 79% 단축**
  - 포맷 변환 및 필요 필드 선별: 약 400초 → 115초
  - 인덱스 기반 조회 구조 적용: 약 115초 → 85초
- 데이터 규모 증가에 따른 클라이언트 **메모리 부담 완화**

기술 스택

React · TypeScript · Redux Toolkit · Tanstack Query · Canvas API · Recharts · Rest API · Websocket · Unity (Unity WebGL) · C# · Python · QGIS · Electron · Git · Jira · Figma